SayangCare: A Fault-Tolerant Stateful Voice Session Manager for Distributed Telehealth

Darren Tan Jun Le, Pieter Imanuel, Chew How Way, Jerlyn Ho, Eng Li Qi, Lucas Ee, Goh Yi Xuan
CSC3104 Cloud and Distributed Computing — AY2026/27, Team 25

Abstract— Telephone-based support can lower the device and interface barriers associated with app-based conversational services, but a voice interaction is stateful and depends on multiple network services. A failed API process or slow language-model provider can interrupt a conversation or lose its context. SayangCare is a Rust and Actix Web prototype that combines Twilio voice webhooks, Redis Sentinel hot session state, PostgreSQL archival, distress-aware circuit breaking, and priority-based volunteer escalation. Its contribution is an integrated reliability design: version-checked session updates and archive-before-delete recovery protect conversational context, while caller distress tightens the inference circuit-breaker thresholds and high-risk turns bypass generative inference for a deterministic safety response and human-review queue. A three-node OpenRaft queue is available as an alternative backend. The current implementation has passed 66 workspace unit tests, including simulated Raft leader-loss recovery. This is functional evidence, not a live-service or performance evaluation: live Twilio end-to-end validation, Kubernetes deployment, load testing, and pod-chaos measurements remain outstanding. SayangCare is a technical prototype, not a clinical or crisis-response service.

Keywords— distributed systems, voice interface, session recovery, circuit breaker, Raft, telehealth triage

1. Introduction and Motivation

Access to digital support is not equally convenient for everyone. A telephone call does not require a user to install an application, navigate a touchscreen, or own a modern smartphone. Voice interfaces and social robots have consequently been investigated as ways to support older adults and people experiencing isolation [7], [8]. That evidence concerns user engagement and wellbeing outcomes; it does not establish that an AI system can provide clinical care. SayangCare therefore treats its conversational component as a prototype for supportive listening and deterministic risk triage, not diagnosis, treatment, or an emergency service.

Telephone access changes the engineering problem. A call may span multiple webhook requests, be processed by different API instances, and depend on a third-party speech and telephony provider and an external inference API. Losing one process must not erase the conversation state, and a slow inference dependency must not hold every caller indefinitely. A conventional stateless request/response service does not by itself solve either issue. Callers at elevated risk also need a path to a human operator rather than an unbounded conversation with a model.

The submitted proposal set out two linked ideas: use session replication and recovery to preserve context across service failures, and adapt circuit-breaker behaviour to conversational distress. The implementation develops those ideas using Redis and PostgreSQL for hot/cold state, optimistic version checks for concurrent updates, an inactivity sweeper, a distress-aware LLM circuit breaker with cached-audio degradation, and a priority queue with auditable volunteer assignments. The project also integrates Twilio webhooks and provides Docker and Kubernetes deployment artefacts.

The contribution is an application-specific combination and evaluation target, not a new consensus protocol or a clinically validated triage algorithm. The design connects caller state, inference degradation, and human escalation in one modular Rust service. It aims to preserve the most recent committed call context and offer a bounded fallback when an optional inference dependency degrades. Whether these mechanisms reduce actual call loss or improve service-level performance has not yet been established experimentally.

2. Related Work

The circuit-breaker pattern isolates a failing dependency by stopping calls after observed failures and allowing a controlled recovery probe [1], [2]. This limits cascading failure and wasted work, but a conventional breaker treats requests according to service health rather than the consequence of failure to a particular user. SayangCare retains the familiar Closed, Open, and Half-Open states, while its adaptive policy lowers error-rate and latency limits as the caller's distress signal increases. The intended effect is to choose a conservative fallback sooner for a caller who may need timely human support. Unlike general-purpose resilience libraries, the policy is connected to a caller/session risk signal; unlike a generic audio fallback, it is designed to preserve the conversation and create an escalation path. Its heuristic signal and thresholds are prototype choices and require validation before operational use.

Raft provides a consensus-based approach to replicated state machines [3]. SayangCare uses the OpenRaft library to replicate queue commands for enqueue, claim, transfer, and release operations. The project does not propose a new Raft algorithm: its engineering contribution is applying a replicated priority queue to human handoff while keeping relational assignment history in PostgreSQL. This separates replicated queue coordination from durable audit records. The Redis sorted-set queue remains the default backend, making the Raft configuration optional rather than a prerequisite for local operation.

Redis Sentinel monitors Redis instances, promotes a replica after master failure, and gives clients a means to discover the current master [4]. This supports a fast hot tier but does not make every session write equivalent to a durable database commit; replication and failover behaviour must be measured under the deployment's consistency requirements. SayangCare therefore keeps a PostgreSQL cold archive and uses archive-before-delete semantics for terminal sessions. This differs from using a cache as the only copy of an active call. It does not, however, guarantee zero loss of an unreplicated last write after Redis failover.

Voice assistants and socially assistive systems have been studied for loneliness and social support, including voice-only modalities [7], [8]. Such work motivates an accessible interaction mode but evaluates human outcomes that SayangCare has not measured. The system focuses instead on distributed reliability and a human-review handoff. It does not claim the benefits reported by those studies, provide a dedicated always-on robot, or demonstrate that its risk classifier is clinically accurate. The distinction is material: SayangCare adds failure-handling and coordination mechanisms around a voice workflow, while prior intervention studies investigate whether a particular device or interaction benefits users.

3. System Design

SayangCare is structured as a modular monolith in a Rust Cargo workspace. The `core` crate defines domain models and service ports; infrastructure crates implement persistence,

telephony, queueing, and circuit breaking; the Actix Web API composes the components and exposes HTTP endpoints. This keeps deployment to one API service while making the interfaces between major responsibilities explicit.

Figure 1. Request and state flow:

```
Caller <-> Twilio <-> Actix API
                      |-- Redis Sentinel: active session state (CAS)
                      |-- Groq inference: distress-aware breaker
                      |      `-- cached audio / deterministic reply
                      |-- Redis queue or OpenRaft: high-risk handoff
                      `-- PostgreSQL: archived calls and assignments
```

3.1 Call and state lifecycle

Twilio sends form-encoded requests to `/twilio/voice`, `/twilio/gather`, and `/twilio/status`; SayangCare returns TwiML rather than JSON on these routes. An incoming `CallSid` becomes the stable session identifier, so webhook retries can resolve to the existing session instead of creating a new conversation. The `/gather` handler records the speech transcript, obtains a response, and returns another gather prompt. The status callback drives terminal archival. Separate JSON endpoints support the local Call Lab, session inspection, and volunteer operations.

Active sessions reside in Redis. Session mutations include a monotonically increasing version and use compare-and-swap (CAS), preventing a stale concurrent request from silently replacing a newer transcript. A Redis miss can recover a record from PostgreSQL. On call completion or abandonment, the system first commits the terminal state, archives the session and JSONB transcript, and only then deletes the matching Redis version. If archival fails, the hot copy is retained for retry; deletion is version-checked so an intervening update is not removed with the older snapshot. A background sweeper scans Redis every 30 seconds and handles sessions inactive for more than five minutes. The scan uses Redis `SCAN`, not a blocking full-key enumeration.

3.2 Inference degradation and risk handling

For non-crisis turns, the API can call a Groq-compatible chat-completion service. Inference calls pass through a sliding-window circuit breaker that records latency and failures, estimates the 95th-percentile latency, and uses Closed, Open, and Half-Open states. A single recovery probe is admitted during Half-Open. Request generations prevent stale completions from changing the state of a newer breaker generation. An adaptive policy scales the error and latency limits according to a normalized distress signal; when inference is unavailable, the service uses a deterministic text reply or a bundled holding-audio response.

Risk classification is a local, weighted phrase heuristic assigning levels 1–5. Levels 4–5 meet the immediate-escalation condition. A high-risk turn bypasses generative inference, receives a fixed safety-oriented response, and is queued for human review. Thus the model is not responsible for deciding whether to persist the session or whether to enqueue a high-risk turn. This reduces dependence on the inference provider for the escalation path, but a phrase matcher can miss paraphrases, context, negation, and multilingual speech. It must not be interpreted as a validated mental-health assessment.

3.3 Queue, assignments, and deployment

The queue interface has two backends. Redis sorted-set queueing is the default; the optional OpenRaft backend replicates deterministic commands and orders unclaimed items by descending risk, then enqueue time, then session identifier. Request identifiers make command retries idempotent. PostgreSQL stores volunteer assignment records and history, with schema constraints intended to prevent two active owners for one session. This pairing provides a volatile operational queue and a separate auditable record, though the in-memory volunteer roster and operator-alert collection remain process-local.

Docker Compose supplies local PostgreSQL, Redis master/replica, Sentinel, and API services. A multi-stage Dockerfile builds the Rust service image. Kubernetes YAML is present for an API Deployment and Service, Redis/Sentinel StatefulSets, PostgreSQL StatefulSet with persistent volume claim, namespace/configuration, and an HPA. These are deployment artefacts rather than evidence of a running cluster. The API image, cluster secrets, ingress/TLS configuration, and custom HPA metric pipeline still require environment-specific configuration and verification.

4. Implementation

The workspace separates six crates: `api`, `core`, `session-store`, `circuit-breaker`, `priority-queue`, and `telephony`. Actix Web applies request tracing, compression, and request identifiers; it exposes the Call Lab dashboard and JSON routes for session and volunteer workflows. The Twilio middleware reconstructs the externally visible URL, sorts form parameters, and verifies the `X-Twilio-Signature` HMAC using the configured auth token. Missing or invalid signatures are rejected. Twilio integration includes Basic-Auth REST calls for call updates and outbound creation, status-callback registration, conference/redirect TwiML, and a signed `/twilio/outbound-connect` route. Real credentials and a public HTTPS callback are required for live calls.

The session-store crate uses SQLx migrations and a PostgreSQL pool for the cold archive. The migration history contains seven files, adding session recovery, handoff state, simulated-session tracking, and assignment/history constraints to the initial schema. Redis stores serialized hot sessions with a one-hour TTL. The tiered store provides hot-first reads, cold-store recovery, terminal-state flush retries with exponential backoff, and version-safe cleanup. The sweeper's five-minute inactivity threshold and 30-second interval are implemented in code. Failed archive writes remain visible in Redis and are retried, rather than being silently treated as successful archival.

The circuit-breaker crate maintains a bounded sliding window, calculates error rate and p95 latency, controls recovery permits, and adjusts configured limits based on distress. The API stores the latest sentiment signal and only uses the inference provider for non-high-risk responses. Holding audio is a versioned local WAV asset served by the API. If a signal-generation call fails while a text reply remains available, the fallback path avoids claiming that the unavailable signal is a successful result.

The priority-queue crate defines the `PriorityQueue` abstraction, Redis implementation, and OpenRaft state machine and transport. The Raft backend supports peer HTTP requests, leader forwarding, snapshots, request-id deduplication, and optional local persistence. It must be explicitly enabled and configured with peer membership; Redis remains the default. Volunteer assignment changes are also written to PostgreSQL, which provides durable history independently of queue membership.

Several proposal targets are only partially implemented. The code offers deterministic risk triage and optional Groq responses, not a validated clinical model. Twilio webhook security and outbound-call code are implemented, but the team's reported tunnel and number configuration have not been corroborated by an end-to-end test record. The repository's `/api/v1/metrics` route currently returns placeholder text; the HPA's `active_voice_sessions` custom metric therefore has no application exporter behind it. Prometheus scraping, a Grafana dashboard, a live Ingress, deployment secrets/image values, and an actual Kubernetes

rollout are not complete. In particular, the checked-in deployment manifest still contains placeholder configuration and needs review before applying to a cluster.

5. Evaluation

The executable evidence available for this report is the workspace unit-test run performed on 14 October 2026. `cargo test --workspace` completed successfully: 66 tests passed, none failed, and one Redis queue test was ignored because it requires an external Redis service. The passed tests cover API fallback and Twilio TwiML paths, session state and risk classification, circuit-breaker transitions and probe concurrency, retry/archive behaviour, queue ordering and claim ownership, Twilio credential guards, and Raft state persistence and replication.

Workspace crate	Passed	Ignored	Representative coverage
API	12	0	Fallback, TwiML, sentiment integration
Circuit breaker	19	0	Sliding window, transitions, recovery permits
Core	16	0	Session state, risk, configuration
Priority queue	8	1	Ordering, ownership, Raft replication/failover
Session store	9	0	CAS, archival retry, recovery
Telephony	2	0	Credential guards
Total	**66**	**1**	**No test failures**

The Raft tests include a simulated three-node queue in which entries replicate and the system continues after leader loss. This is evidence about the in-process test harness and Raft state-machine implementation, not a Kubernetes pod-failure experiment. Session-store tests exercise temporary archive failures, version conflicts, retries, and retention of the hot record after retry exhaustion. These tests support the correctness of individual code paths; they do not quantify durability under real Redis/PostgreSQL outages or network partitions.

No k6 load run, 100-plus-user benchmark, real phone-call test, production-like Kubernetes deployment, Prometheus collection, or pod-termination chaos test is recorded in the repository at this snapshot. Consequently there are no defensible throughput, p95/p99 response-time, resource-use, call-drop, recovery-time, or cross-pod consistency measurements to report. A placeholder metrics endpoint is not measurement evidence. The one ignored Redis integration test was not counted as passed. Reported setup of the Twilio number and ngrok tunnel is team-provided progress status, not a substitute for logged call outcomes.

The evaluation supports a limited conclusion: important state transitions and fault-handling rules have unit-level coverage, including a simulated Raft failover, and the full workspace test command passes. It does not support claims about end-to-end availability or performance. A complete evaluation should exercise complete simulated call lifecycles at increasing concurrency, report latency distributions and error rates, and repeat controlled API-pod terminations while calls are active. Each run should record environment, replica counts, Redis failover events, archived session versions, queue consistency, and losses or retries. Live Twilio validation should separately record caller/volunteer call status and webhook error codes, using consenting testers and configured verified numbers.

6. Discussion and Limitations

The design's principal reliability choice is to treat the conversation as durable, versioned state rather than state owned by one HTTP process. Redis supports active-call access, PostgreSQL supplies an archive, and CAS plus archive-before-delete reduces the chance that a stale handler or transient database outage erases the latest known session. The design also distinguishes operational queue coordination from relational audit history. These boundaries are appropriate for a distributed prototype, but are not a proof of linearizability across Redis failover, PostgreSQL, and the Raft queue. In particular, a crash between queue insertion and session update can leave the two representations temporarily inconsistent; recovery and reconciliation must be tested.

The service is a modular monolith rather than a set of independently deployed microservices. This reduces local orchestration cost and keeps latency-sensitive operations within one process, while Redis, PostgreSQL, Twilio, and optional Raft nodes remain distributed dependencies. The trade-off is that API replicas share dependencies and process-local roster/alert state does not automatically synchronize. Production deployment would need externalized roster/alert storage, least-privilege secret management, secure network policies, configured ingress/TLS, and readiness checks that reflect dependency health.

Clinical and operational safety boundaries are equally important. Keyword risk scoring is not a substitute for a trained and independently evaluated risk model or a human crisis service. Synthetic sessions and deterministic test responses do not demonstrate safe real-caller outcomes. Any pilot would need a documented escalation protocol, consent and data-retention policy, privacy review for sensitive transcript storage, accessibility testing, and clear user messaging about the service's non-clinical scope. The current report therefore makes no claim that SayangCare is ready to handle real mental-health emergencies.

7. Conclusion and Future Work

SayangCare implements a voice-session prototype that combines signed Twilio webhooks, hot/cold persistence, recoverable session updates, distress-aware inference degradation, and priority-based human handoff. Its main contribution is the integration of user-context-sensitive degradation with state recovery and queue-backed escalation in a containerized distributed application. Workspace tests provide initial correctness evidence, including simulated Raft replication and leader loss, but no live telephony, cluster deployment, or performance conclusions are available yet.

The next steps are to complete the Prometheus exporter and dashboard, reconcile the HPA metric with a real metrics adapter, finish and validate Kubernetes configuration including public Ingress and secrets, and run the planned concurrency and pod-failure tests. The team should then update this article with measured results, rerun live Twilio tests with consenting participants, review the risk heuristic and safety procedure, and revise the claims to match the evidence before final submission.

References

[1] M. Fowler, "CircuitBreaker," *martinfowler.com*, 2014. [Online]. Available: https://martinfowler.com/bliki/CircuitBreaker.html

[2] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.

[3] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annual Technical Conference (USENIX ATC '14)*, 2014, pp. 305–319. [Online]. Available: https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro

[4] Redis Ltd., “High availability with Redis Sentinel.” [Online]. Available: https://redis.io/docs/latest/operate/oss_and_stack/management/sentinel/

[5] Twilio, “Security: Validating requests.” [Online]. Available: https://www.twilio.com/docs/usage/security#validating-requests

[6] Kubernetes Authors, “Horizontal Pod Autoscaling.” [Online]. Available: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

[7] V. K. Jones, C. Yan, M. Y. Shade, et al., “Reducing loneliness and improving social support among older adults through different modalities of personal voice assistants,” *Geriatrics*, vol. 9, no. 2, p. 22, 2024, doi: 10.3390/geriatrics9020022.

[8] Y. Yang, C. Wang, X. Xiang, and R. An, “AI applications to reduce loneliness among older adults: A systematic review of effectiveness and technologies,” *Healthcare*, vol. 13, no. 5, p. 446, 2025, doi: 10.3390/healthcare13050446.